In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn import metrics
from sklearn.metrics import classification_report, confusion_matrix, roc_curve, auc, accuracy_score, precision_
```

In [2]:
```
dtree = pd.read_csv('bill_authentication.csv')
display(dtree.head())
```

	Variance	Skewness	Curtosis	Entropy	Class
0	3.62160	8.6661	-2.8073	-0.44699	0
1	4.54590	8.1674	-2.4586	-1.46210	0
2	3.86600	-2.6383	1.9242	0.10645	0
3	3.45660	9.5228	-4.0112	-3.59440	0
4	0.32924	-4.4552	4.5718	-0.98880	0

In [3]:
```
# Basic Info and Statistics
print("Dataset Info:")
dtree.info()
print("\nDataset Description:")
dtree.describe()
```

```
Dataset Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1372 entries, 0 to 1371
Data columns (total 5 columns):
 #   Column    Non-Null Count  Dtype  
---  ------    --------------  -----  
 0   Variance  1372 non-null   float64
 1   Skewness  1372 non-null   float64
 2   Curtosis  1372 non-null   float64
 3   Entropy   1372 non-null   float64
 4   Class     1372 non-null   int64  
dtypes: float64(4), int64(1)
memory usage: 53.7 KB

Dataset Description:
```

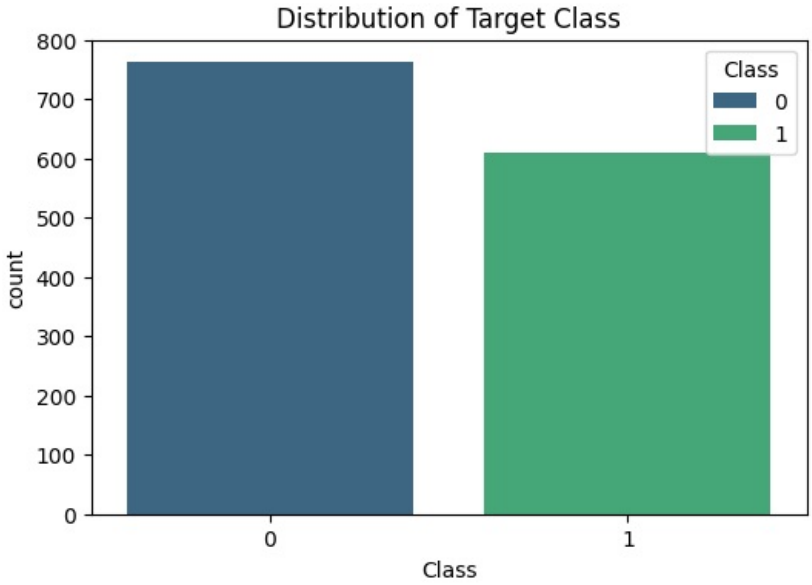
Out[3]:

	Variance	Skewness	Curtosis	Entropy	Class
count	1372.000000	1372.000000	1372.000000	1372.000000	1372.000000
mean	0.433735	1.922737	1.397627	-1.191657	0.444606
std	2.842763	5.868619	4.310030	2.101013	0.497103
min	-7.042100	-13.773100	-5.286100	-8.548200	0.000000
25%	-1.773000	-1.708200	-1.574975	-2.413450	0.000000
50%	0.496180	2.319650	0.616630	-0.586650	0.000000
75%	2.821475	6.814625	3.179250	0.394810	1.000000
max	6.824800	12.951600	17.927400	2.449500	1.000000

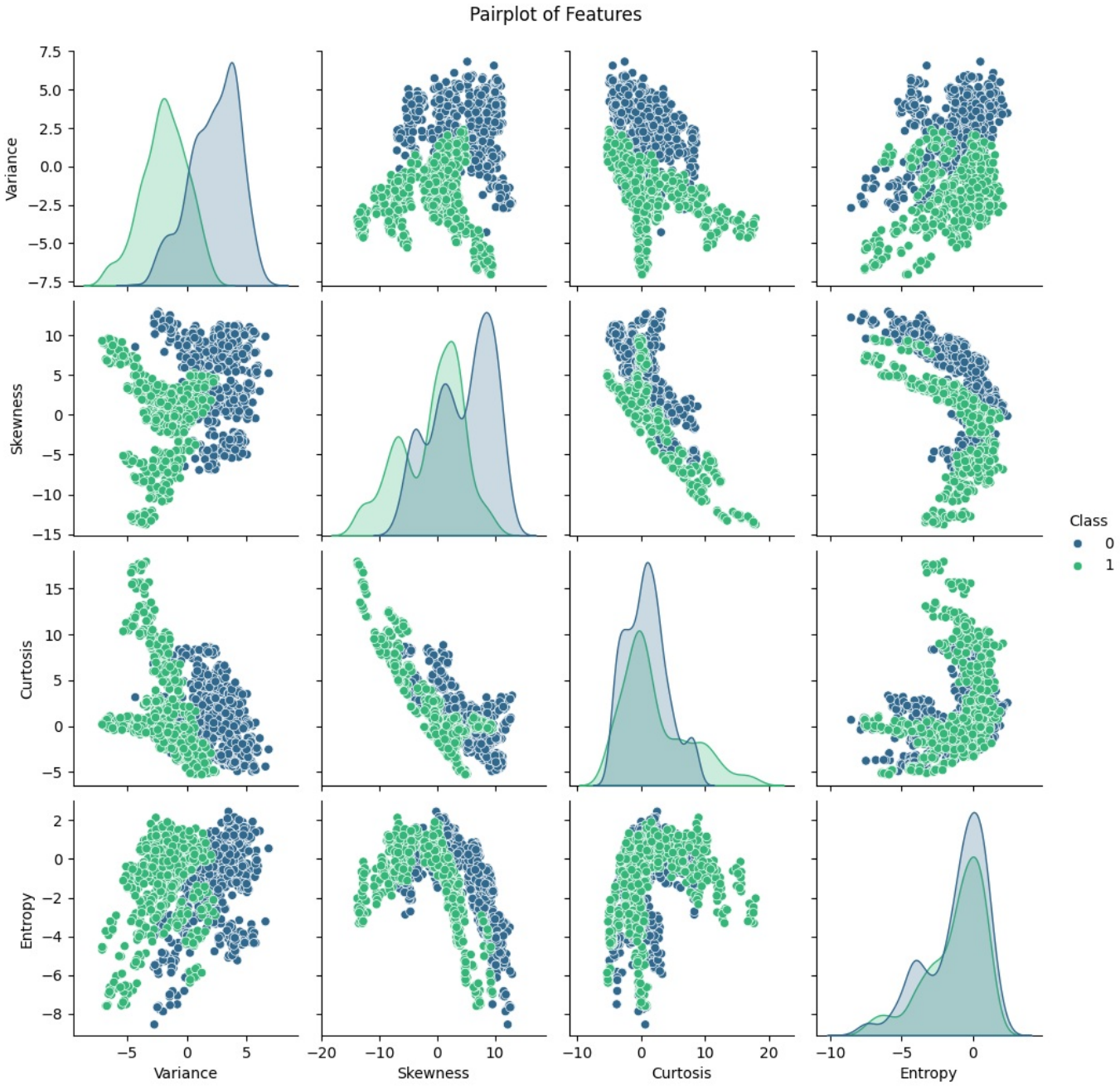
In [4]:
```
# Check for missing values
print("Missing Values:")
print(dtree.isnull().sum())
```

```
Missing Values:
Variance    0
Skewness    0
Curtosis    0
Entropy     0
Class       0
dtype: int64
```

In [5]:
```
# Distribution of Target Class
plt.figure(figsize=(6,4))
sns.countplot(x='Class', data=dtree,hue='Class', palette='viridis')
plt.title('Distribution of Target Class')
plt.show()
```

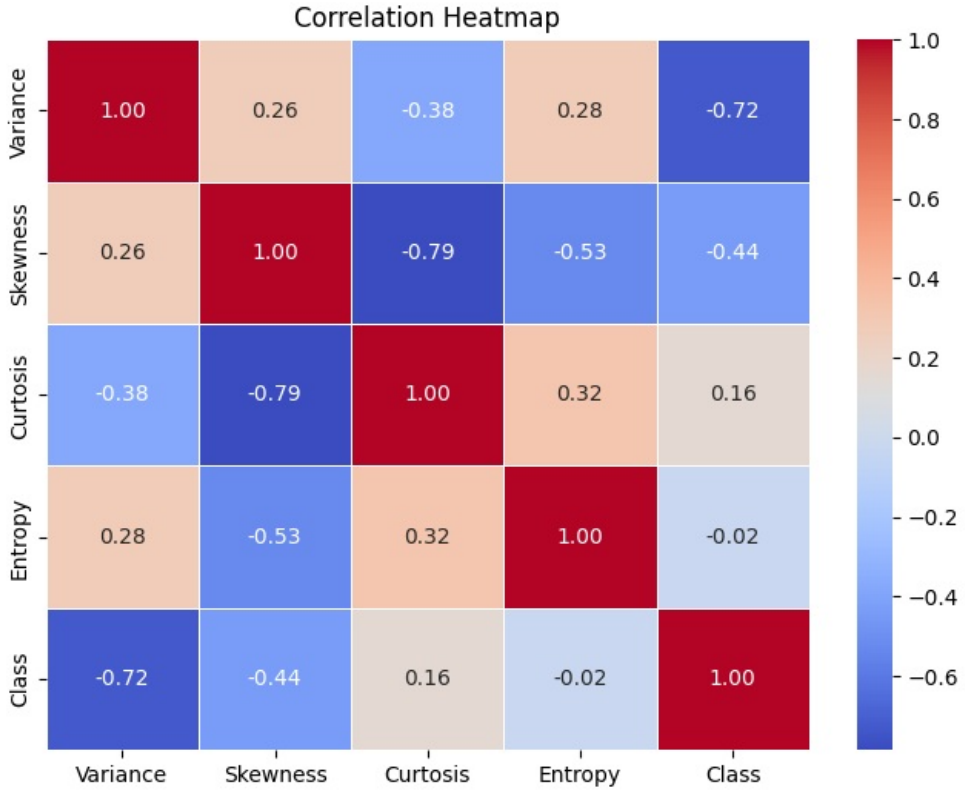


In [6]:
```
# Pairplot for Feature Distributions and Relationships
sns.pairplot(dtree, hue='Class', palette='viridis', diag_kind='kde')
plt.suptitle('Pairplot of Features', y=1.02)
plt.show()
```



In [7]:
```
# Correlation Heatmap
plt.figure(figsize=(8,6))
sns.heatmap(dtree.corr(), annot=True, cmap='coolwarm', fmt=".2f", linewidths=0.5)
```

```
plt.title('Correlation Heatmap')
plt.show()
```



In [8]:
```
X = dtree.drop('Class', axis=1)
y = dtree['Class']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20, random_state=42)
print(f"Training set shape: {X_train.shape}")
print(f"Testing set shape: {X_test.shape}")
```

```
Training set shape: (1097, 4)
Testing set shape: (275, 4)
```

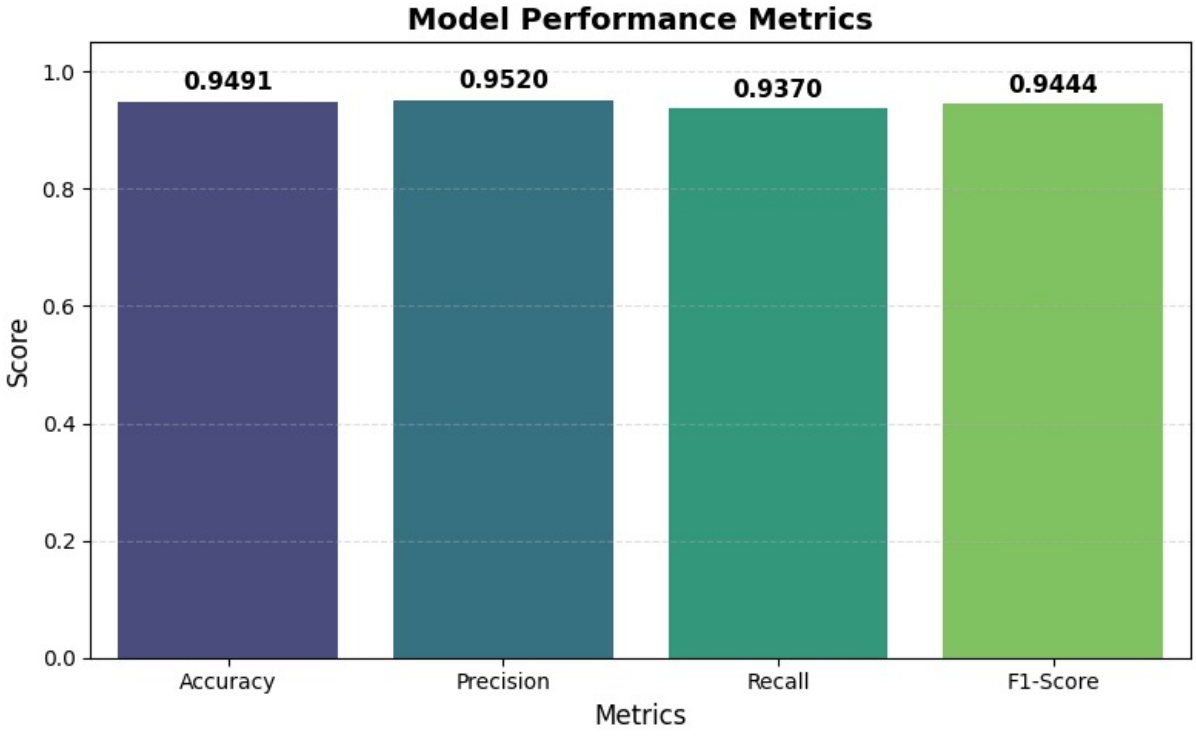
In [9]:
```
# Initialize and train Decision Tree
clf = DecisionTreeClassifier(criterion='entropy', max_depth=3, random_state=42)
clf.fit(X_train, y_train)
y_pred = clf.predict(X_test)
y_prob = clf.predict_proba(X_test)[:, 1] # Probabilities for ROC curve
```

In [10]:
```
# Calculate Metrics
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)

print("--- Model Performance Metrics ---")
print(f"Accuracy : {accuracy:.4f}")
print(f"Precision: {precision:.4f}")
print(f"Recall   : {recall:.4f}")
print(f"F1-Score : {f1:.4f}\n")

print("Classification Report:")
print(classification_report(y_test, y_pred))
```

```
--- Model Performance Metrics ---
Accuracy : 0.9491
Precision: 0.9520
Recall   : 0.9370
F1-Score : 0.9444

Classification Report:
              precision    recall  f1-score   support

           0       0.95      0.96      0.95       148
           1       0.95      0.94      0.94       127

    accuracy                           0.95       275
   macro avg       0.95      0.95      0.95       275
weighted avg       0.95      0.95      0.95       275
```

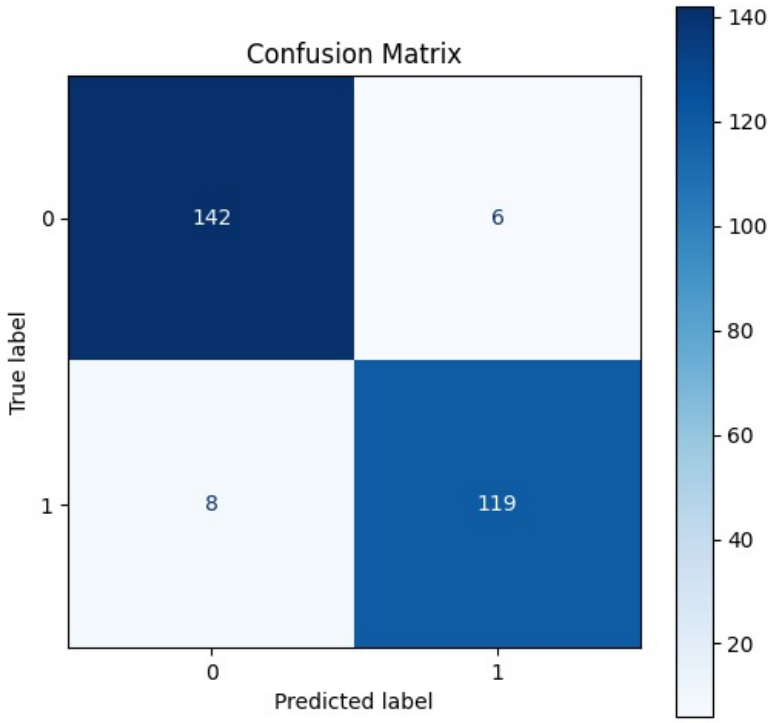
In [11]:
```
# Visualizing Classification Metrics
```

```
metrics_dict = {
    'Accuracy': accuracy,
    'Precision': precision,
    'Recall': recall,
    'F1-Score': f1
}

# Create figure
plt.figure(figsize=(8, 5))

# Bar Plot
ax = sns.barplot(
    x=list(metrics_dict.keys()),
    y=list(metrics_dict.values()),
    hue=list(metrics_dict.keys()),
    palette='viridis',
    legend=False
)

# Axis limits and labels
plt.ylim(0, 1.05)
plt.xlabel("Metrics", fontsize=12)
plt.ylabel("Score", fontsize=12)
plt.title('Model Performance Metrics', fontsize=14, fontweight='bold')

# Add value labels on bars
for i, v in enumerate(metrics_dict.values()):
    ax.text(
        i,
        v + 0.02,
        f"{v:.4f}",
        ha='center',
        fontsize=11,
        fontweight='bold'
    )

# Add grid for better readability
plt.grid(axis='y', linestyle='--', alpha=0.4)

# Improve layout
plt.tight_layout()

# Show plot
plt.show()
```

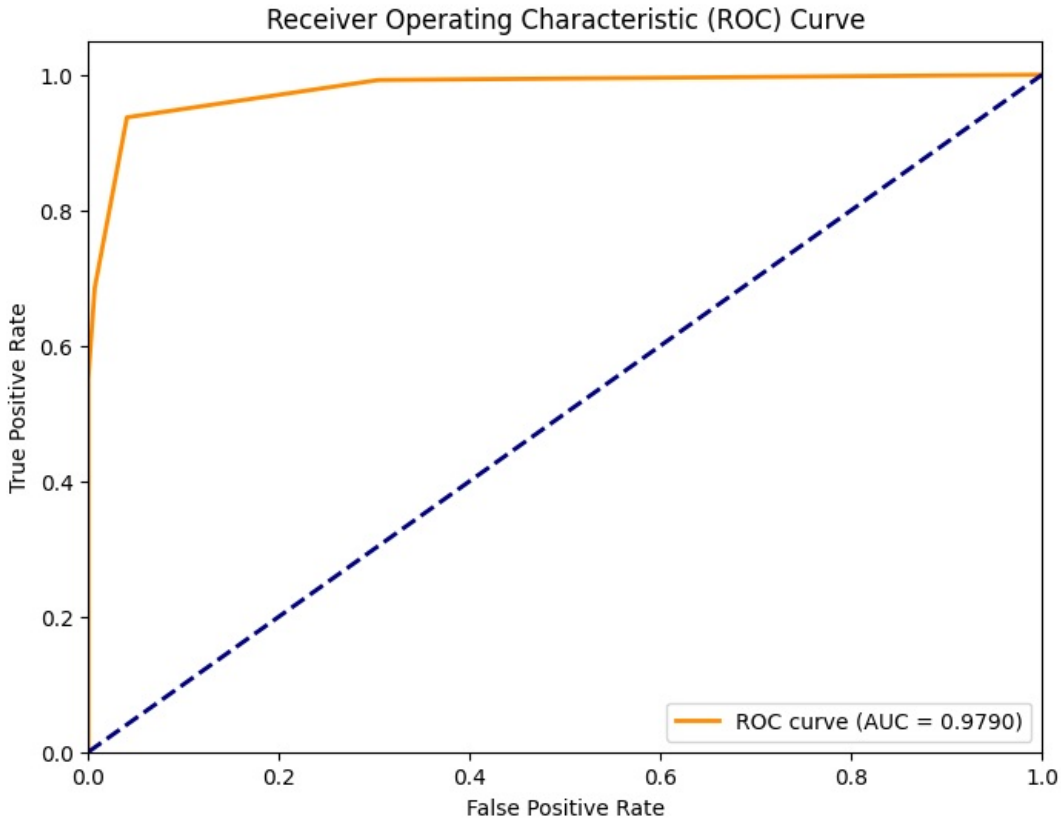


In [12]:

```
# Confusion Matrix Visualization
cm = confusion_matrix(y_test, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=clf.classes_)

fig, ax = plt.subplots(figsize=(6, 6))
disp.plot(cmap='Blues', ax=ax, values_format='d')
plt.title('Confusion Matrix')
plt.show()
```

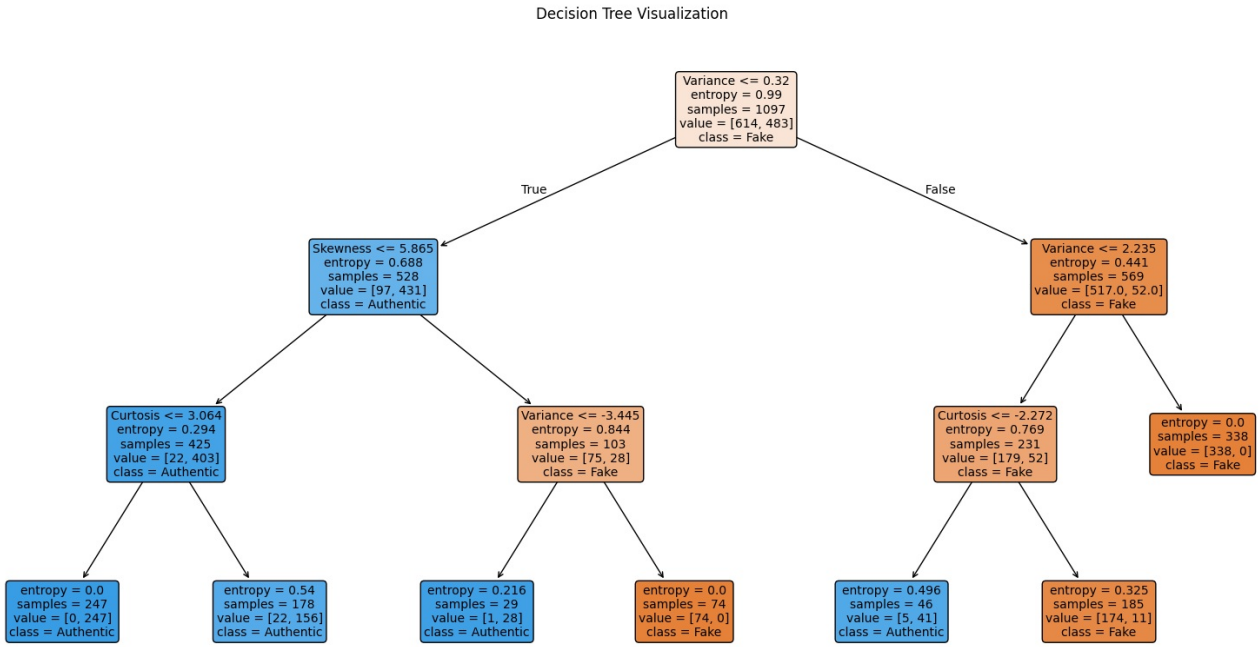


In [13]:
```
# ROC Curve and AUC
fpr, tpr, thresholds = roc_curve(y_test, y_prob)
roc_auc = auc(fpr, tpr)

plt.figure(figsize=(8, 6))
plt.plot(fpr, tpr, color='darkorange', lw=2, label=f'ROC curve (AUC = {roc_auc:.4f})')
plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic (ROC) Curve')
plt.legend(loc="lower right")
plt.show()
```



In [14]:
```
plt.figure(figsize=(20, 10))
plot_tree(clf,
          feature_names=X.columns,
          class_names=['Fake', 'Authentic'],
          filled=True,
          rounded=True,
```

```
          fontsize=10)
plt.title('Decision Tree Visualization')
plt.show()
```



In [15]:
```
# Example prediction on a new data point
import pandas as pd

# Define the data
sample_data = [[3.45660, 9.5228, -4.0112, -3.59440]]

# Convert to DataFrame with the same column names used during training to avoid warnings
sample_df = pd.DataFrame(sample_data, columns=X.columns)

prediction = clf.predict(sample_df)
prob = clf.predict_proba(sample_df)

class_name = 'Authentic' if prediction[0] == 1 else 'Fake'
print(f"Sample Data: {sample_data[0]}")
print(f"Predicted Class: {prediction[0]} ({class_name})")
print(f"Prediction Probabilities: Fake={prob[0][0]:.4f}, Authentic={prob[0][1]:.4f}")
```

```
Sample Data: [3.4566, 9.5228, -4.0112, -3.5944]
Predicted Class: 0 (Fake)
Prediction Probabilities: Fake=1.0000, Authentic=0.0000
```